Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет “Высшая школа экономики”

Факультет компьютерных наук
Дисциплина «Архитектура компьютера и операционные системы»

ОТЧЁТ
по индивидуальному заданию № 3

Вариант 31
«Управление взлётно-посадочными полосами»

Выполнил: Шепурев Степан Сергеевич
Группа: БПИ252
Дата: 10.11.2026

Москва, 2026

Содержание

1 Постановка задачи и модельные решения

2 Структура программы и роли процессов

3 Синхронизация и взаимодействие процессов

4 Диспетчеризация и обеспечение корректности

5 Пользовательский интерфейс и ввод данных

6 Сборка запуск и завершение программы

7 Тестирование

8 Результаты демонстрационных запусков

9 Критическая оценка и использование ИИ

Приложение А События и состав проекта

Работа посвящена процессной реализации варианта 31. Представлены устройство программы, четыре способа взаимодействия процессов, правила работы аэропорта, результаты проверок и ограничения текущей версии.

1 Постановка задачи и модельные решения

Цель индивидуального задания № 3 — разработать консольную имитационную программу для Linux на языке C с использованием libc, системных вызовов и взаимодействующих процессов. Из ДЗ2 сохранены модель аэропорта и формат настроек; потоки заменены процессами.

Условие варианта 31

Аэропорт располагает одной или несколькими взлётно-посадочными полосами. Самолёты прибывают через случайные интервалы и запрашивают посадку или взлёт. Каждый самолёт имеет уникальный номер, тип, требуемую длину полосы, продолжительность операции и категорию приоритета.

Прибывающий самолёт, не получивший полосу, остаётся в зоне ожидания и расходует топливо. Вылетающий ожидает на земле. Диспетчер назначает совместимую свободную полосу с учётом интервала безопасности. Критический остаток топлива даёт аварийный приоритет, но уже начатая операция не прерывается.

Входные данные задают полосы, типы и количество самолётов, интервалы запросов, длительность операций, топливо, приоритеты и закрытия. Вывод содержит запросы, очередь, изменение топлива, назначения, начала и окончания операций, освобождения полос, аварийный статус, уход на запасной и закрытия.

Принятые допущения

Время дискретное: такт соответствует условной единице времени. Длительности, интервалы и расход топлива задаются в тактах. Топливо означает расходуемый резерв ожидания; при его исчерпании регистрируется DIVERT. Это учебная модель, а не физический расчёт перелёта на запасной аэродром.

Каждый самолёт, каждая полоса и диспетчер представлены самостоятельными процессами. Дополнительный координатор продвигает время и собирает журнал. FLOW генерирует конечный список заявок; MAX_TICKS=0 отключает предел смены, но не отменяет завершение после обработки всех заявок.

Время продвигается после подтверждений участников. Шаги состояния выполняются по очереди, а операции на разных полосах могут перекрываться в модельном времени. Задержка вывода не изменяет длительность операции в тактах. Seed фиксирует генерацию входных данных в пределах одной реализации libc.

2 Структура программы и роли процессов

Файл	Назначение
src/airport.h	Структуры самолёта, полосы и Airport; общие объявления.
src/main.c	Параметры запуска, настройки, журнал и итоговый код выхода.
src/config.c	Чтение сценария, проверка входа, генерация FLOW.
src/model.c	Правила самолётов, полос и диспетчера; инварианты и статистика.
src/simulation.c	Порядок фаз такта, часы и условия завершения.
src/ipc.c	Процессы, общая память, FIFO, семафоры и очистка.
src/io.c	Оформление событий и запись через дескрипторы.
CMakeLists.txt	Сборка четырёх версий и универсального airport.

Соответствие участников процессам

Участник	Поведение
Каждый самолёт	Подаёт заявку, входит в очередь, обновляет топливо и аварийный статус.
Каждая ВПП	Обрабатывает закрытия, начинает и завершает назначенную операцию.
Диспетчер	Выбирает ожидающие самолёты и резервирует совместимые свободные полосы.
Координатор	Организует запуск, порядок фаз, часы, сигналы, журнал и остановку.

Для N самолётов и R полос создаётся N + R + 1 процессов-участников. Вместе с координатором одновременно работают N + R + 2 процесса. В основном сценарии N=30 и R=3: 34 участника и координатор. ID 0..N−1 принадлежат самолётам, следующие R ID — полосам, последний ID — диспетчеру.

Все данные объединены в Airport. В версиях с общей памятью процессы обращаются к одному объекту mmap. В FIFO-версии Airport передаётся по каналу. Ограничения: 300 самолётов, 10 полос, 20 типов, 30 закрытий и 20 потоков FLOW. Состояние изменяет один разрешённый участник, остальные ждут семафоры.

3 Синхронизация и взаимодействие процессов

Дочерние процессы и общая память

airport_named создаёт участников через fork. Данные хранятся в объекте POSIX shared memory, созданном shm_open и отображённом mmap с MAP_SHARED. У каждого участника есть именованный семафор gate. Общий ack подтверждает обработку шага, ready — подключение участника.

airport_unnamed использует ту же модель и общую память. Семафоры расположены внутри отображённого объекта и создаются sem_init с pshared=1. Эта версия реализует второй обязательный вариант взаимодействия дочерних процессов.

Координатор разрешает работу одному участнику через sem_post и ждёт ack. До ответа следующий участник не получает доступ к изменению модели. После завершения дочерние процессы собираются waitpid; неименованные семафоры уничтожаются sem_destroy, именованные закрываются и удаляются.

Независимые процессы и каналы FIFO

airport_shm с --external создаёт общую память и именованные семафоры, затем ожидает независимо запущенных участников. Команда --worker shm SESSION ID подключает один процесс к существующей сессии. В этом режиме координатор не вызывает fork.

airport_fifo с --external передаёт данные через два именованных канала FIFO, созданных mkfifo. Разделяемая память не создаётся. Именованные POSIX-семафоры задают адресата и подтверждение. Обычные read/write передают Airport с учётом частичного чтения и записи. Каналы разрешены требованиями уровня выше 8 баллов.

Сценарий взаимодействия

Один такт состоит из четырёх фаз: самолёты регистрируют запросы и обновляют топливо; полосы завершают операции и обрабатывают закрытия; диспетчер назначает полосы; полосы начинают назначенные операции. Затем проверяются инварианты и продвигаются часы.

worker_loop ждёт gate, выполняет actor_step и подаёт ack. По ID выбирается plane_step, runway_step либо dispatcher_step. Для независимого запуска предусмотрен скрипт run_independent.py; он запускает участников как отдельные процессы, соседние координатору. Можно выполнить те же команды вручную.

Имена объектов включают UID и имя сессии. O_EXCL предотвращает перезапись существующих объектов, права 0600 ограничивают доступ владельцем. Семафор claim препятствует повторному подключению одного ID. На подключение всех участников отведено 20 секунд; обмен имеет тайм-аут.

4 Диспетчеризация и обеспечение корректности

Состояния самолёта и правила назначения

```
PENDING -> WAITING -> ASSIGNED -> RUNNING -> DONE
             |           |
             v           +-> WAITING при отмене резерва
          DIVERTED
```

ASSIGNED означает резервирование полосы. Перед START повторно проверяются топливо и закрытие. Если резерв отменён, самолёт возвращается в WAITING либо получает DIVERT. После начала операции состояние RUNNING сохраняется до её завершения или остановки моделирования.

Диспетчер сравнивает аварийный статус, затем HIGH > NORMAL > LOW, затем request_tick и порядок во входных данных. Выбирается самая короткая доступная полоса достаточной длины. Новое назначение не прерывает начатую операцию.

После FINISH на такте T устанавливается next_free_tick = T + safety. Закрытия задаются полуинтервалами [start, end). Полоса может завершить начатую операцию во время закрытия, но не начинает новую, пока закрытие действует.

Топливо прибывающего уменьшается на burn × число прошедших тактов ожидания. Повторный расчёт в том же такте не списывает его второй раз. При fuel ≤ burn × EMERGENCY_FUEL_TICKS включается аварийный приоритет. При fuel=0 регистрируется DIVERT; отрицательный остаток не допускается.

Инварианты

Требование	**Способ обеспечения**
Не более одного самолёта на ВПП	Один plane_index, последовательные назначения и проверка взаимных ссылок.
Совместимость по длине	Проверка при выборе полосы и в check_invariants.
Интервал безопасности	Запрет назначения до next_free_tick; проверка START/RELEASE в тестах.
Операция не повторяется	Переходы состояний и times_started ≤ 1.
Явный исход по топливу	EMERGENCY и DIVERT; остаток ограничивается снизу нулём.

check_invariants проверяет взаимные ссылки самолёта и полосы, совместимость, неотрицательное топливо и times_started ≤ 1. Интервал безопасности обеспечивается выбором полосы и дополнительно проверяется по журналу в тестах. Нарушение инварианта приводит к INVARIANT_FAILURE.

5 Пользовательский интерфейс и ввод данных

Интерфейс состоит из командной строки, файла конфигурации и журнала. По умолчанию читается configs/default.conf, затем применяются значения CLI. Разбор и генерация самолётов завершаются до запуска участников. Универсальная программа airport позволяет выбрать режим через --mode.

Параметр	Назначение
--config, -c	Путь к конфигурации.
--seed	Начальное значение генератора.
--max-ticks	Последний такт смены; 0 — без ограничения.
--delay-ms	Задержка между тактами в миллисекундах.
--log / --no-log	Файл журнала либо отключение записи.
--quiet / --help	Тихий вывод либо справка.
--mode	named, unnamed, shm или fifo.
--session	Имя сессии для объектов IPC.
--external	Ожидать независимо запускаемых участников.
--worker MODE NAME ID	Подключить процесс-участник к сессии.

Пример конфигурации

```
SEED 31
MAX_TICKS 30
DELAY_MS 100
EMERGENCY_FUEL_TICKS 3
RUNWAY R1 2500 2
TYPE A320 1800 3 3
CLOSE R1 10 15
AIRCRAFT A001 ARRIVAL 0 A320 12 1 NORMAL
AIRCRAFT D001 DEPARTURE 1 A320 0 0 HIGH
```

RUNWAY задаёт имя, длину и интервал безопасности; TYPE — имя, требуемую длину и длительность операций. AIRCRAFT задаёт номер, операцию, такт запроса, тип, топливо, расход и приоритет. CLOSE содержит имя полосы и границы закрытия.

FLOW задаёт операцию, число самолётов, первый такт и диапазон интервалов. PRIORITY_WEIGHTS, ARRIVAL_FUEL и ARRIVAL_BURN определяют случайные параметры. Имена должны быть уникальными; ссылки на полосу или тип следуют после их объявления.

Ввод и вывод

Конфигурация открывается через open и читается fdopen/fgets. strtol проверяет числа: отвергаются лишние символы, переполнение и недопустимые значения. Проверяются повторные имена, неизвестные типы и полосы, число элементов и формат строк.

Журнал открывается open и записывается write с учётом EINTR и частичной записи. Участник помещает события в Airport, координатор выводит их по порядку. Строка содержит такт, тег и описание; в конце печатаются SUMMARY и состояния незавершённых самолётов.

6 Сборка запуск и завершение программы

Требуются Linux, компилятор C11 и CMake 3.16 или новее. На Windows используется Ubuntu через WSL. Python 3 нужен для автоматических тестов и удобного запуска независимых процессов; сама модель реализована на C.

Все команды выполняются из корня проекта в Linux. Для получения журналов демонстраций и результатов проверок используются следующие команды:

```
cmake -S . -B build
cmake --build build -j4
./build/airport_named --log logs/demo.log
python3 scripts/run_independent.py --mode fifo \
  -c configs/test_nominal.conf --log logs/independent_fifo.log
python3 tests/run_tests.py
```

Для второго базового варианта используется airport_unnamed. Независимая версия с общей памятью запускается через run_independent.py --mode shm. Для ручного запуска координатору передают --external --session NAME, а каждому участнику — --worker shm|fifo NAME ID. Диапазоны ID выводятся в READY.

Условия завершения

Условие	Наблюдаемый результат
Все самолёты DONE или DIVERTED	COMPLETE; ALL_REQUESTS_COMPLETED; код 0.
Достигнут MAX_TICKS > 0	SHIFT-END; SHIFT_ENDED; код 0; незавершённые заявки в итогах.
SIGINT или SIGTERM координатору	STOP; USER_INTERRUPT; код 130 в проверенных обычных сценариях.
Неверный ввод / открытие журнала	Диагностика, код 2; модель не запускается.
Ошибка IPC или инварианта	IPC_FAILURE / INVARIANT_FAILURE; код 3.
Ошибка записи / закрытия журнала	Ошибка ввода-вывода; код 4.

Если подходящей полосы нет, прибывающий расходует топливо и получает DIVERT; вылетающий ждёт до окончания смены либо сигнала. При MAX_TICKS=0 такой вылет может ждать неограниченно. Завершение смены не объявляет выполнявшуюся операцию успешно законченной.

Остановка и освобождение ресурсов

Координатор блокирует SIGINT и SIGTERM и проверяет их через sigtimedwait. Участники игнорируют SIGINT, чтобы Ctrl+C всей группе оставлял координатору возможность завершить обмен. При обычной остановке каждому отправляется STOP, затем собираются дочерние процессы и удаляются объекты IPC.

При ошибке координатор посылает дочерним процессам SIGTERM. Семафоры закрываются и удаляются, общая память освобождается, FIFO удаляются из /tmp. Частично созданные объекты учитываются отдельно. У обработки зависших участников и подтверждений завершения есть известные недостатки, приведённые в разделе 9.

7 Тестирование

Сборка и проверки выполнены в Ubuntu под WSL2, GCC 15.2.0, с -Wall -Wextra -Wpedantic -Werror. Все 54 проверки tests/run_tests.py прошли. Реальные процессы запускаются во всех четырёх режимах; shm и fifo проверяются с независимым запуском. Результаты записаны в report/test_results.txt.

Сценарий	Что проверяется	Результат
test_nominal.conf	Две посадки и два взлёта.	Четыре версии прошли.
test_emergency.conf	Аварийный A001 раньше D002.	Четыре версии прошли.
test_closure.conf	CLOSE/OPEN, отсутствие START внутри закрытия.	Четыре версии прошли.
test_incompatible.conf	DIVERT и ожидающий вылет к концу смены.	Четыре версии прошли.
test_safety.conf	START не раньше доступности из RELEASE.	Четыре версии прошли.
test_empty / test_zero_fuel	Пустой аэропорт / нулевое топливо.	Четыре версии прошли.
test_duplicate.conf	Отклонение дубликата, код 2.	Четыре версии прошли.
SIGINT / SIGTERM	Выход и удаление ресурсов.	Проверки прошли.
Гибель участника	Ошибка IPC, выход остальных, уборка.	shm и fifo прошли.
Одинаковый seed	Сравнение событий четырёх версий.	Последовательности совпали.
SIGSTOP участника	Выход координатора после тайм-аута.	Зависание named и unnamed.

Границы проверок

54 проверки охватывают основные сценарии и несколько способов остановки, но не доказывают отсутствие всех ошибок. При дополнительной проверке с SIGSTOP дочернего процесса версии named и unnamed зависли в waitpid после тайм-аута. Ctrl+C в этот момент не завершил координатор; после SIGCONT работа завершилась.

В коде также выявлено чтение command->stop после отправки ack. В общей памяти координатор уже может изменить команду, что создаёт риск гонки при остановке. Это замечание получено при анализе кода; отдельного воспроизведения проявления гонки в текущем наборе тестов нет.

Ошибка финального обмена STOP не возвращается из функции очистки в основной цикл: причина завершения может остаться штатной. Требуется добавить проверки остановленного участника, передачи подтверждения и ошибок завершающего обмена. Найденные дефекты в текущем коде ещё не исправлены.

В отчёте используются фактические результаты текущей процессной версии. Проверки и числа из прежней многопоточной работы не считаются подтверждением ДЗ3. Совпадение событий разных IPC-версий проверяется при одинаковом seed и одной конфигурации.

8 Результаты демонстрационных запусков

В logs/demo.log сохранён запуск airport_named с configs/default.conf: 30 самолётов, 3 полосы, 5 типов и два окна закрытия. В logs/independent_fifo.log сохранён независимый FIFO-запуск test_nominal.conf: 4 самолёта и 2 полосы. Это разные входные сценарии; их показатели приведены для демонстрации двух способов запуска.

Показатель	**demo.log: named**	**independent_fifo.log**
Сценарий	default.conf	test_nominal.conf
Причина завершения	Все заявки обработаны	Все заявки обработаны
Последний такт	63	9
Всего самолётов	30	4
Посадки / взлёты	10 / 14	2 / 2
Уходы на запасной	6	0
Ожидающие / активные / будущие	0 / 0 / 0	0 / 0 / 0
Назначения / аварийные	24 / 1	4 / 0
Операции по полосам	9 / 10 / 5	2 / 2
Процессы-участники	34 дочерних	7 независимых

В основном сценарии баланс исходов: 10 посадок + 14 взлётов + 6 уходов на запасной = 30. В независимом FIFO-сценарии: 2 посадки + 2 взлёта = 4. Незавершённых самолётов нет. Число операций полос совпадает с числом завершённых посадок и взлётов.

Фрагмент журнала независимого FIFO-запуска

```
[T=0000] [ASSIGN   ] ATC assigned D001 to R-LONG
[T=0000] [ASSIGN   ] ATC assigned A001 to R-SHORT
[T=0000] [START    ] D001 started TAKEOFF on R-LONG; finish at T=4
[T=0000] [START    ] A001 started LANDING on R-SHORT; finish at T=3
[T=0003] [FINISH   ] A001 completed LANDING on R-SHORT
[T=0004] [FINISH   ] D001 completed TAKEOFF on R-LONG
```

Фрагмент показывает перекрытие операций на двух полосах в модельном времени. Шаг каждой полосы выполняет её процесс. Обмен общей структурой через FIFO не использует разделяемую память.

Тесты дополнительно сравнивают полные последовательности событий всех четырёх версий на default.conf. При одинаковом seed они совпадают: порядок фаз фиксирован и не зависит от скорости пробуждения участников.

9 Критическая оценка и использование ИИ

Использование ИИ и внесённые изменения

При подготовке проекта использовался: для анализа задания, тестов, проверки результатов и подготовки документации. Запросы в report/AI_PROMPTS.md.

По сравнению с первой работой последовательные вызовы ролей заменены самостоятельными pthread. Добавлены world, уведомления участников и отдельное состояние ASSIGNED, отделяющее резерв от начала операции. Подготовлены два исполняемых файла с общей логикой и различными примитивами ожидания.

Поведение участников выделено в plane_step, runway_step и dispatcher_step. Цикл моделирования вынесен в simulation.c, системное взаимодействие находится в ipc.c. FIFO заменяет более сложный вариант очередей сообщений и сохраняет разрешённый требованиями способ обмена для верхнего уровня.

Ограничения и необходимые улучшения

Изменения сериализованы координатором, а в FIFO передаётся всё состояние Airport. Это упрощает учебную модель, но увеличивает накладные расходы. Асинхронные уведомления процессов сочетаются с фиксированными фазами; схему необходимо объяснить на защите. Сетевое взаимодействие и переносимость бинарного формата не реализованы.

Перед сдачей требуется исправить зависание при остановленном дочернем процессе, сохранить признак STOP до ack и передавать ошибки финальной очистки в основной цикл. SIGKILL и сбой питания не позволяют выполнить штатную уборку; для остатков остановленной сессии есть cleanup_session.py. Четыре вида IPC реализованы, но оценки 9–10 не гарантируются.

Приложение А События и состав проекта

Событие	Тег журнала
Запрос посадки или взлёта	REQUEST
Постановка в очередь	QUEUE
Изменение топлива	FUEL
Назначение полосы	ASSIGN
Начало операции	START
Окончание операции	FINISH
Освобождение полосы	RELEASE
Аварийный приоритет	EMERGENCY
Уход на запасной аэродром	DIVERT
Закрытие и открытие полосы	CLOSE / OPEN
Возврат после отмены резерва	REQUEUE
Штатное завершение / смена / сигнал	COMPLETE / SHIFT-END / STOP

Состав проекта

```
airport_runway_hw3/
  CMakeLists.txt, README.md
  src/
    airport.h, main.c, config.c, io.c
    model.c, simulation.c, ipc.c
  configs/
    default.conf, test_*.conf
  scripts/
    run_independent.py, run_tests.sh, cleanup_session.py
  tests/
    run_tests.py
  logs/
    demo.log, independent_fifo.log
  report/
    REPORT.md, REPORT.pdf
    AI_PROMPTS.md, test_results.txt
```

Каталог build создаётся при сборке и не является исходным кодом. Для защиты нужно объяснить четыре вида IPC, роли участников, порядок такта, выбор полосы, топливо и ограничения завершения. Подробные команды приведены в README.md.